

DOC Intelligence — Documento de Especificação Técnica

Documento: SPEC-001
Módulo: Ingestão e Processamento de Documentos (Trilha A — Back-end)
Versão: 0.1.0
Status: Rascunho para implementação
Data: 30/08/2026

1. Objetivo

Este documento especifica o comportamento, as restrições e os critérios de aceitação da fatia vertical do serviço DOC Intelligence a ser implementada nesta entrega. Não especifica o produto completo descrito no desafio original, apenas o subconjunto definido na Seção 2.

Convenções de linguagem normativa usadas neste documento: deve indicar requisito obrigatório; não deve indicar proibição obrigatória; deveria indicar recomendações fortes, com desvio permitido mediante justificativa registrada em ADR; PODE indica comportamento opcional.

2. Escopo

2.1 Escopo desta entrega

O sistema deve implementar o seguinte fluxo, de ponta a ponta:

ID	Capacidade
CAP-01	Receber um documento (imagem ou PDF) via API síncrona, com resposta imediata
CAP-02	Validar o tipo real do arquivo, independentemente de metadados fornecidos pelo cliente
CAP-03	Deduplicar documentos por conteúdo, de forma segura sob concorrência
CAP-04	Processar o documento de forma assíncrona, com resiliência a falhas do provedor de extração
CAP-05	Registrar o resultado da extração com proveniência completa (modelo, prompt, parâmetros)
CAP-06	Rotear o resultado por limiar de confiança, sem permitir que um resultado incerto seja tratado como definitivo
CAP-07	Permitir consulta do status e do resultado de um documento, individualmente e em lista

2.2 Fora de escopo desta entrega

Os itens abaixo são reconhecidos como parte do produto-alvo, mas não fazem parte desta entrega. Cada um é tratado por uma das duas formas abaixo, indicada na tabela:

Item	Tratamento	Justificativa
Reivindicação (<i>claim</i>) de documento para conferência humana	Especificado em contrato (Seção 5.3), não implementado	Pertence à fronteira com a interface do atendimento (Trilha B), fora do recorte escolhido
Consumo por sistemas internos de terceiros	Decisão de mecanismo registrada em ADR-004, não implementado	Não há sistema consumidor real nesta fase; simular um consumidor fictício não agrega valor de engenharia
Autenticação e autorização reais	Placeholder de API-key estática, sinalizado como não-produtivo	Fora do critério de aceitação da fatia vertical
Criptografia em repouso e política de retenção/expurgo de dados pessoais	Registrado como risco conhecido (Seção 6)	Extrapolando o tempo disponível para a fatia vertical; tratado como débito técnico explícito
Deploy em ambiente produtivo	Fora de escopo	Não solicitado pelo desafio

3. Premissas e Restrições do Ambiente

As restrições abaixo são fatos observados sobre o ambiente de produção-alvo, não requisitos negociáveis. Cada uma condiciona um ou mais requisitos funcionais e não-funcionais definidos nas Seções 4 e 5.

ID	Restrição	Requisitos derivados
ENV-A	O provedor de extração é um modelo de linguagem multimodal de terceiro; latência de 5–40s por chamada; sujeito a erro ou ausência de resposta	RF-04, RF-05, RNF-01

ENV-B	A origem do documento (aplicação de atendimento) não valida tipo, nome ou conteúdo do arquivo antes do envio	RF-02
ENV-C	O mesmo documento pode ser reenviado múltiplas vezes pela mesma origem ou por origens distintas	RF-03
ENV-D	O conteúdo processado constitui dado pessoal, incluindo dado pessoal sensível	RNF-04
ENV-E	Volume médio de 150 documentos/dia, com picos superiores a 800, concentrados em janela de 2 horas	RNF-02
ENV-F	O modelo e os prompts do provedor de extração estão sujeitos a alteração de versão ao longo do primeiro ano	RF-05, ADR-002
ENV-G	Múltiplos operadores podem tentar acessar a fila de conferência concomitantemente	Especificado, não implementado (Seção 2.2)

4. Requisitos Funcionais

RF-01 – Ingestão

O sistema deve expor um endpoint de ingestão que aceite um arquivo binário e retorne, de forma síncrona, um identificador do documento e seu status inicial, sem aguardar o processamento completo.

RF-02 – Validação de tipo

O sistema deve determinar o tipo real do arquivo por inspeção de conteúdo (assinatura binária), e não deve confiar em nome de arquivo, extensão ou Content-Type declarado pelo cliente para essa determinação. Arquivos fora da whitelist de formatos suportados devem ser rejeitados na ingestão, antes de qualquer enfileiramento.

RF-03 – Deduplicação

O sistema deve calcular um hash de conteúdo (SHA-256) para cada documento recebido. Se um documento com hash idêntico já existir, o sistema não deve reprocessá-lo, e deve retornar o identificador e status do documento já existente. Esta verificação deve ser segura sob concorrência (duas ingestões simultâneas do mesmo conteúdo não devem resultar em duas chamadas ao provedor de extração).

RF-04 – Processamento assíncrono e resiliência

O processamento deve ocorrer de forma desacoplada da ingestão, via fila. Em caso de falha do provedor de extração, o sistema deve distinguir falhas retentáveis (timeout, erro 5xx, ausência de resposta) de falhas não-retentáveis (erro de validação de entrada, recusa do modelo), aplicando retentativa com espera exponencial apenas ao primeiro grupo. Um

documento que esgote as retentativas não deve permanecer indefinidamente em estado de processamento; deve transicionar para um estado terminal de falha, roteável para conferência humana.

RF-05 — Proveniência do resultado

Todo resultado de extração deve ser gravado junto com um registro imutável da configuração que o produziu (identificador do modelo, texto do prompt utilizado, parâmetros relevantes de inferência).

RF-06 — Roteamento por confiança

O sistema deve aplicar um limiar de confiança configurável ao resultado da extração. Resultados abaixo do limiar não devem ser marcados como estado terminal de sucesso; devem ser roteados a um estado de pendência de revisão.

RF-07 — Consulta

O sistema deve permitir a consulta do status e do resultado de um documento por identificador, e a listagem de documentos filtrável por status.

5. Requisitos Não-Funcionais

RNF-01 — Resiliência a dependência externa instável

O sistema deve permanecer operacional (aceitando novas ingestões) mesmo quando o provedor de extração estiver integralmente indisponível.

RNF-02 — Capacidade

A concorrência do worker de processamento deve ser configurável externamente (variável de ambiente ou equivalente), sem exigir alteração de código para ajuste sob pico de volume.

RNF-03 — Modularidade e substituíbilidade

A integração com o provedor de extração deve estar isolada do restante do domínio por uma interface (porta), de modo que a substituição do provedor ou de sua versão não exija alteração em outras camadas do sistema.

RNF-04 — Tratamento de dado sensível

Nenhum registro de log estrutural deve conter, em texto plano, os campos extraídos de um documento.

RNF-05 — Testabilidade

Os três pontos de risco identificados nas premissas do ambiente (RF-03 sob concorrência, RF-04 quanto à diferenciação de erro, RF-06 quanto à não-degradação de resultado incerto) devem possuir cobertura de teste automatizado. Cobertura ampla e não-direcionada não é requisito desta entrega.

6. Riscos Conhecidos e Não Mitigados

ID	Risco	Motivo de não mitigação nesta entrega
RISK-01	Dado pessoal sensível armazenado sem criptografia em repouso	Fora do tempo disponível para a fatia vertical; requer decisão de infraestrutura (gestão de chaves) não trivial
RISK-02	Ausência de política de retenção/expurgo de dados processados	Depende de definição jurídica (prazo de retenção) não fornecida no desafio; pergunta enviada ao avaliador, sem resposta até o momento da entrega
RISK-03	Ausência de mecanismo de <i>claim/lease</i> para concorrência na fila de conferência (ENV-G)	Pertence à Trilha B; contrato especificado, implementação fora de escopo
RISK-04	Deduplicação (ADR-003) só detecta bytes idênticos; quase-duplicatas (mesma foto tirada duas vezes, ângulo/compressão diferentes) não são identificadas e são processadas como documentos novos	Resolver com segurança exige hash perceptual calibrado com dados reais, tratamento diferenciado para PDF (requer renderização prévia da página) e uma política definida para o caso de falso positivo, que, neste domínio, tem custo mais alto que um reproprocessamento redundante (risco de misturar dado de duas pessoas). Ver ADR-003 para a análise técnica completa

7. Rastreabilidade

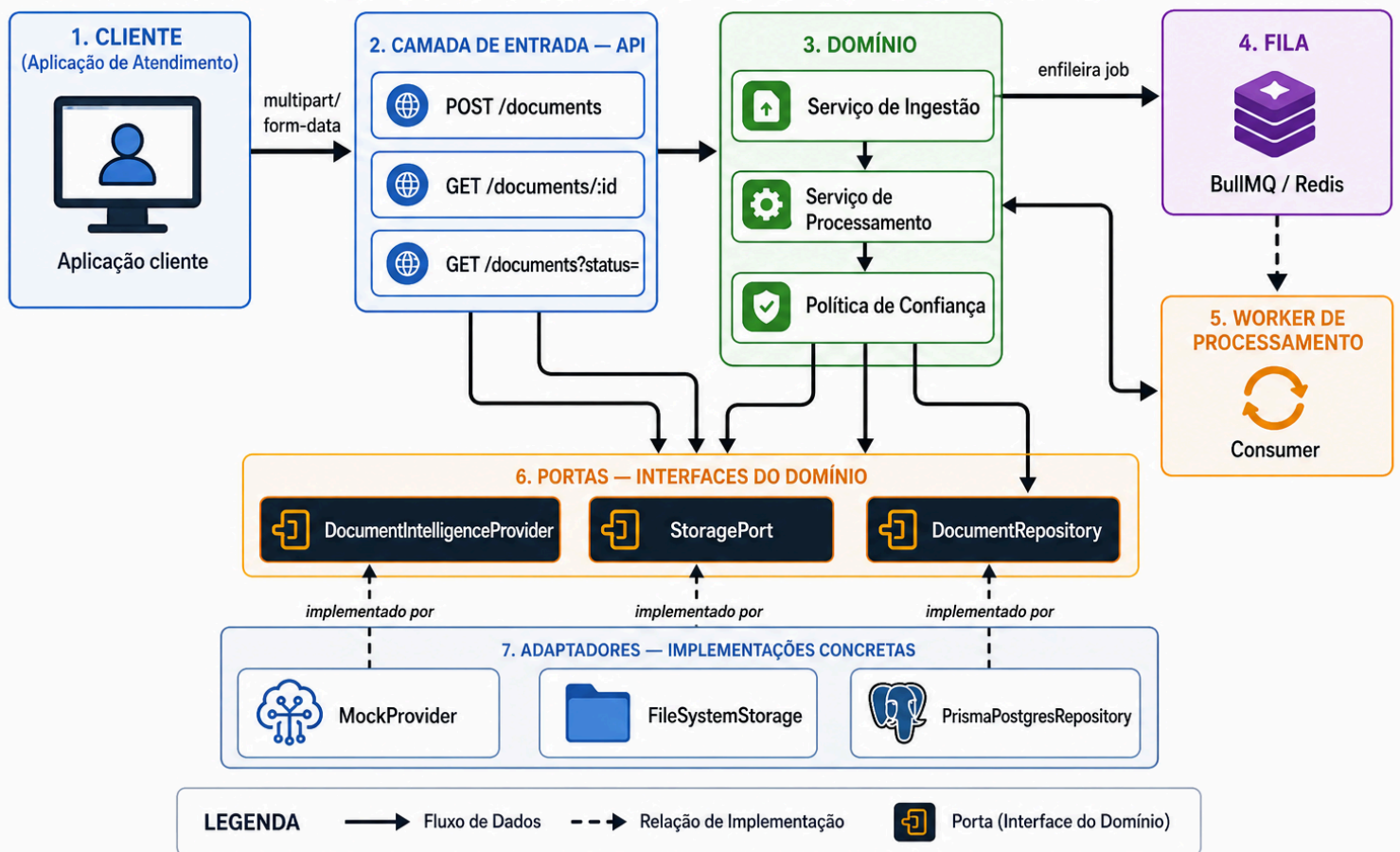
A matriz completa de rastreamento entre fatos do ambiente (Seção 3), requisitos (Seções 4–5) e decisões de arquitetura será mantida nos registros de decisão (ADR-001 a ADR-00N), referenciados individualmente à medida que forem redigidos.

8. Modelagem e Diagramas de Arquitetura

8.1 Diagrama de Componentes:

O diagrama mostra como o sistema é organizado por dentro, dividindo-o em blocos com responsabilidades bem separadas.

DIAGRAMA DE COMPONENTES ARQUITETURA DE PROCESSAMENTO DE DOCUMENTOS



1. Cliente (Aplicação de Atendimento)

É quem começa tudo; a aplicação usada pelo atendimento do escritório. **Entra:** nada, é o ponto de partida. **Sai:** o arquivo do documento, enviado como multipart/form-data (formato padrão para envio de arquivos pela internet).

2. Camada de Entrada (API)

São as três "portas de entrada e saída" que o mundo externo usa pra falar com o sistema:

- **POST /documents** — recebe um arquivo novo.
- **GET /documents/:id** — pergunta o status de um documento específico.
- **GET /documents?status=** — pergunta a lista de documentos com um certo status.

Entra: requisições vindas do Cliente. **Sai:** encaminha para o bloco 3 (Domínio) e devolve as respostas para quem perguntou.

3. Domínio

É o "cérebro" do sistema, onde as regras de negócio de verdade acontecem, em três etapas, em sequência:

- **Serviço de Ingestão** — recebe o arquivo, confere se é válido, calcula a impressão digital (hash) e decide se é documento novo ou repetido.
- **Serviço de Processamento** — organiza a chamada para a inteligência artificial que vai ler o documento.
- **Política de Confiança** — decide se o resultado que a IA devolve é bom o suficiente para confiar, ou se precisa de revisão humana.

Entra: o arquivo validado pela API. **Sai:** ou manda pra fila (documento novo) ou já responde que existe (documento repetido); mais tarde, recebe de volta o resultado do processamento vindo do Worker.

4. Fila (BullMQ/Redis)

Funciona como uma esteira de espera. O Domínio deposita aqui o aviso "tem um documento para processar", e esse aviso fica esperando até que o Worker esteja livre. **Entra:** o "job" (aviso de trabalho) enviado pelo Serviço de Ingestão. **Sai:** entrega esse job pro Worker quando ele estiver disponível.

5. Worker de Processamento

É o trabalhador (o "Consumer") que fica de olho na fila, pega um job por vez, e aciona de volta o Serviço de Processamento (bloco 3) para tocar a extração: o Domínio manda pra fila, e o Worker chama de volta o Domínio quando pega o job.

6. Portas — Interfaces do Domínio

São as três "tomadas elétricas" que o Domínio usa pra falar com o mundo de fora, sem saber os detalhes de quem está do outro lado:

- **DocumentIntelligenceProvider** — tomada para conversar com a IA que lê o documento.
- **StoragePort** — tomada para guardar/buscar o arquivo original.
- **DocumentRepository** — tomada para gravar/consultar dados no banco.

Entra: os pedidos vindos do Domínio (setas sólidas de "Fluxo de Dados"). **Sai:** repassa esses pedidos para o bloco 7, e devolve a resposta pro Domínio.

7. Adaptadores — Implementações Concretas

São os "plugues" reais que encaixam em cada tomada do bloco 6, nesta entrega específica, ligação marcada no diagrama pela seta tracejada "implementado por":

- **MockProvider** - encaixa no DocumentIntelligenceProvider (substituto fake da IA de verdade).
- **FileSystemStorage** - encaixa no StoragePort (guarda o arquivo em disco).
- **PrismaPostgresRepository** - encaixa no DocumentRepository (grava no banco Postgres de verdade).

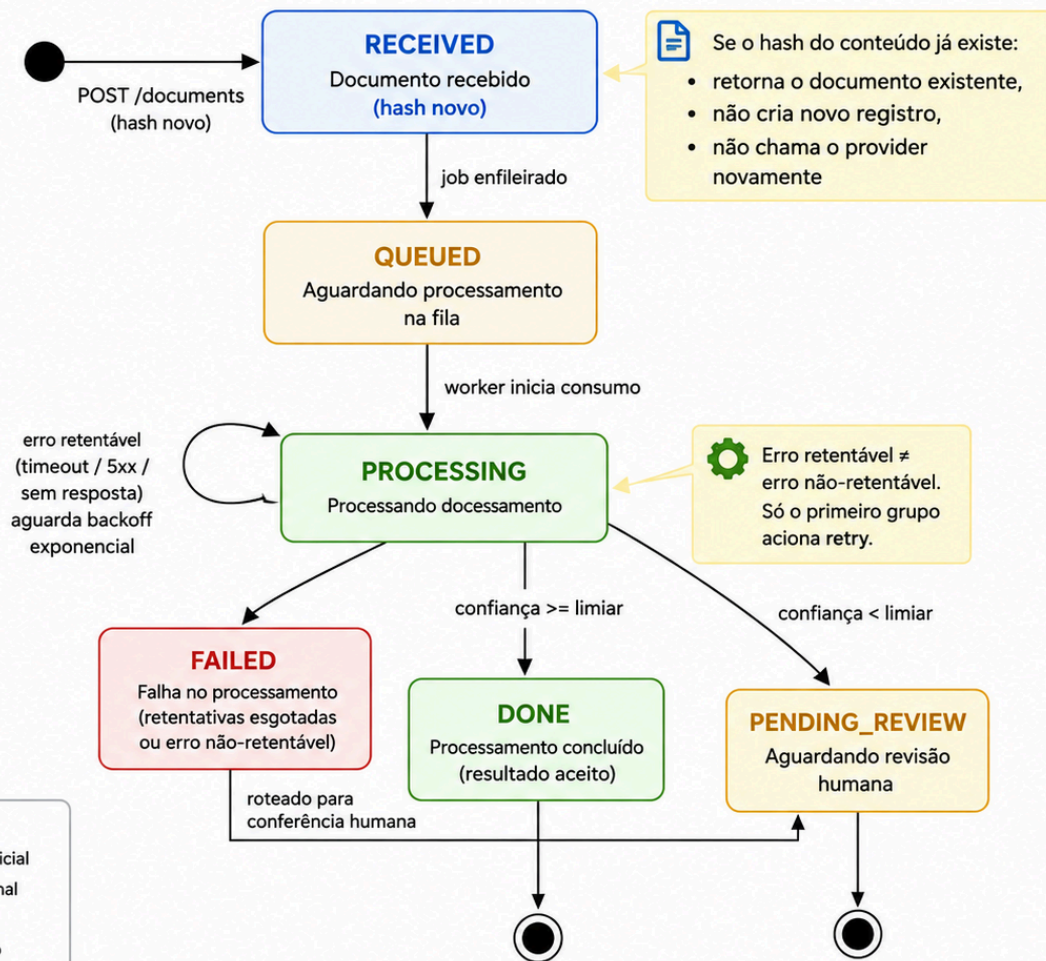
Entra: o pedido que veio da porta correspondente. **Sai:** a ação concreta realizada, arquivo salvo, dado gravado, resposta simulada da IA.

8.2 Diagrama de Máquina de Estados:

Esse diagrama representa a "vida" de um único documento dentro do sistema, não é sobre computadores conversando entre si, e sim sobre as fases pelas quais um documento específico passa, desde o momento em que alguém envia uma foto ou PDF até o momento em que ele fica pronto para ser consultado.

Pense nele como o rastreamento de uma encomenda: "postado" → "em trânsito" → "entregue", só que aqui é "recebido" → "processando" → "pronto" (ou "precisa de revisão"). Cada caixa é uma etapa possível, e cada seta mostra qual mudança de etapa é permitida.

Diagrama de Máquina de Estados — Processamento de Documentos



Estado inicial:

É o ponto de partida, o "começo da história". Não é uma etapa em si, é onde tudo começa: alguém enviou um documento novo.

Caixa azul — RECEIVED (Documento recebido):

O sistema acabou de receber o documento e confirmou que ele é realmente novo (nunca tinha visto esse arquivo antes). A nota amarela ao lado explica uma regra importante: se alguém mandar o mesmo documento de novo (o mesmo arquivo, reenviado por engano ou por precaução), o sistema percebe que já tem esse documento, devolve a informação que já existe, e não gasta tempo nem dinheiro processando de novo. Isso evita trabalho duplicado.

Caixa laranja — QUEUED (Aguardando na fila):

O documento está numa espécie de "fila de espera", como numa fila de banco. Ele já foi recebido, mas ainda não chegou a vez dele ser efetivamente analisado.

Caixa verde — PROCESSING (Processando):

Agora sim, alguém (no caso, um programa automatizado) está analisando o documento, tentando descobrir que tipo de documento é e extrair as informações dele. Repare na seta

que sai e volta pra essa mesma caixa, do lado esquerdo: isso representa uma tentativa de novo em caso de falha temporária, por exemplo, se o serviço de análise demorou demais pra responder ou deu um erro passageiro, o sistema tenta de novo automaticamente, esperando um pouquinho mais a cada tentativa (é o que a legenda chama de "aguardar" backoff exponencial", simplesmente "espera cada vez mais entre uma tentativa e outra"). A nota amarela à direita reforça que isso só acontece para erros temporários, se o erro for de um tipo que nunca vai se resolver tentando de novo, o sistema não insiste à toa.

A partir daqui, existem três caminhos possíveis, dependendo do que aconteceu na análise:

Caixa vermelha — FAILED (Falha no processamento):

O sistema tentou várias vezes e não conseguiu, ou encontrou um erro que não adianta tentar de novo. Mesmo assim, o documento não é descartado, ele é encaminhado para uma pessoa revisar manualmente (seta "roteado para conferência humana").

Caixa verde — DONE (Processamento concluído):

Deu tudo certo: o sistema conseguiu extrair as informações do documento e tem confiança suficiente de que o resultado está correto. O documento está pronto, sem necessidade de revisão humana.

Caixa laranja — PENDING_REVIEW (Aguardando revisão humana):

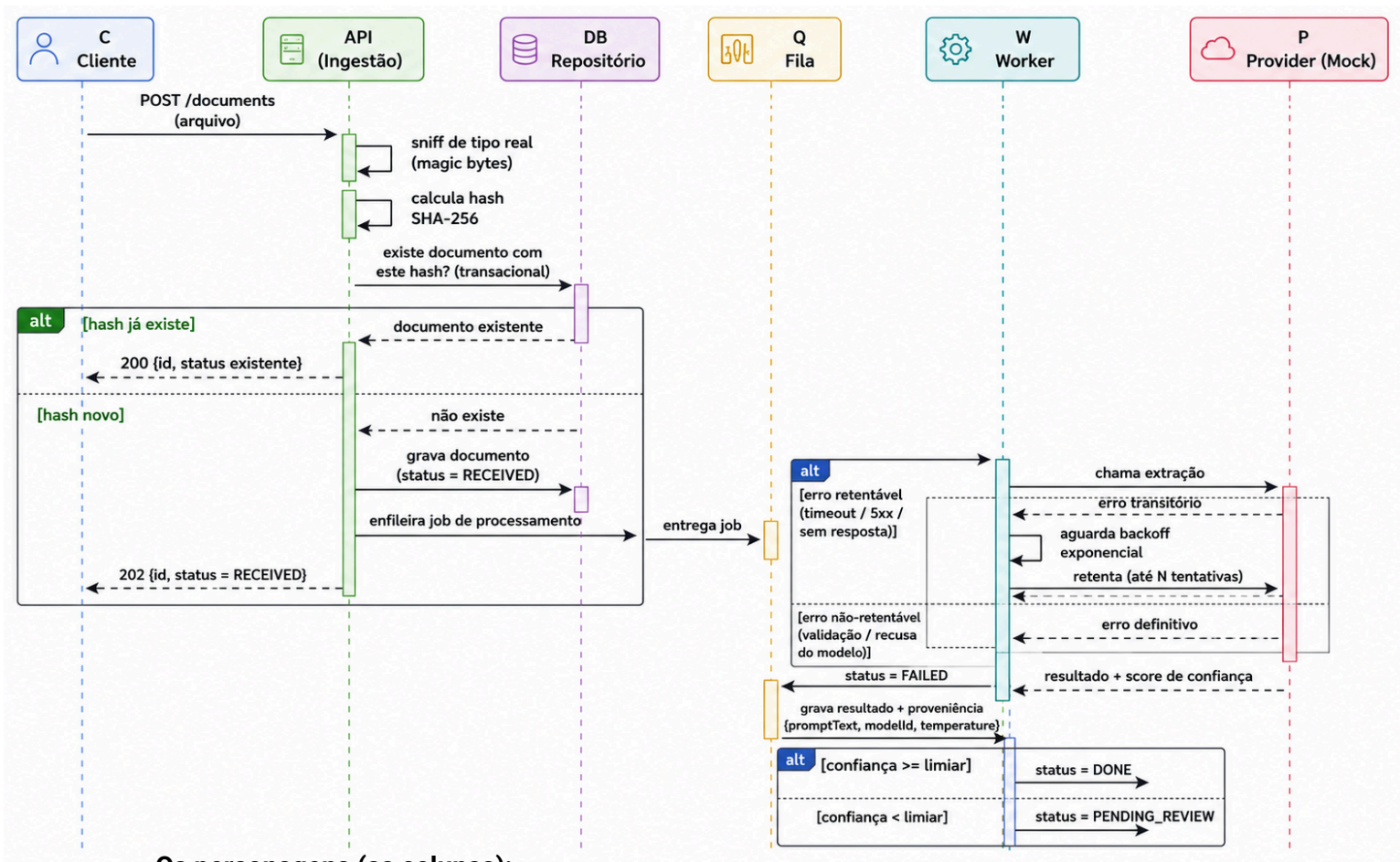
O sistema até conseguiu processar o documento, mas o resultado não é confiável o bastante (por exemplo, a foto estava borrada ou o texto ficou ambíguo). Nesse caso, mesmo sem ter "falhado" tecnicamente, o documento é enviado para uma pessoa conferir antes de considerar pronto.

Estados finais:

São os dois pontos de chegada possíveis dessa "história": ou o documento chegou pronto (via DONE) e será consultado normalmente, ou ele entrou na fila de revisão humana (via FAILED ou PENDING_REVIEW) — repare que as duas caixas se encontram no mesmo destino de revisão humana antes de finalizar.

8.3 Diagrama de Sequência — Fluxo Principal:

Este documento mostra a conversa passo a passo entre as diferentes partes do sistema, na ordem exata em que ela acontece, para processar um único documento do início ao fim. Pense nele como um roteiro de teatro: cada coluna vertical é um "personagem" (uma parte do sistema), e as setas horizontais são as falas — quem manda uma mensagem pra quem, e em que ordem. É esse diagrama que prova, na prática, como as etapas do diagrama anterior (RECEIVED, PROCESSING, DONE...) realmente acontecem por trás dos panos.



Os personagens (as colunas):

- **Cliente (C)** – quem está enviando o documento (a aplicação do atendimento).
- **API (Ingestão)** – a "porta de entrada" que recebe o pedido e faz as primeiras verificações.
- **DB Repositório** – o banco de dados, onde tudo fica guardado.
- **Fila (Q)** – a esteira de espera.
- **Worker (W)** – o trabalhador que pega o documento da fila e toca o processamento.
- **Provider (P)** – o serviço de inteligência artificial (aqui, uma versão "de mentirinha" usada só para testes, por isso "Mock").

A conversa, passo a passo

1. O envio

O Cliente manda o arquivo para API (POST /documents). A API faz duas checagens rápidas nela mesma: primeiro confere qual é o tipo real do arquivo (não confia só na extensão do nome do arquivo) abre e olha o conteúdo de verdade, depois calcula uma espécie de "impressão digital" única daquele arquivo (o hash).

2. A pergunta ao banco de dados

A API pergunta ao Repositório: "esse arquivo, pela impressão digital dele, já existe aqui?". Essa pergunta acontece de um jeito seguro mesmo se duas pessoas mandarem o mesmo arquivo ao mesmo tempo (é o que a nota "transacional" indica).

3. A bifurcação — bloco cinza "alt" (alternativa)

Esse bloco retangular com a etiqueta "alt" representa uma escolha: só uma das duas situações dentro dele vai acontecer, nunca as duas.

- Se o hash já existe: o Repositório avisa a API, e a API simplesmente responde ao Cliente "olha, esse documento já está aqui" (resposta 200), sem processar nada de novo. Documento repetido não gera trabalho duplicado.
- Se o hash é novo: o Repositório confirma que não existe, a API manda gravar o documento com status "recebido", e então avisa a Fila que tem um trabalho novo para fazer. A API já responde ao Cliente "recebi, aqui está o número dele" (resposta 202), sem esperar o processamento terminar, porque isso pode demorar.

4. A fila entrega o trabalho ao Worker

A Fila guarda esse aviso até que o Worker esteja livre, e então o entregue. É a mesma lógica de "esteira de espera" do diagrama anterior.

5. A tentativa de extração — outro bloco "alt"

O Worker pede ao Provider (a IA) para analisar o documento. Aqui tem outra bifurcação:

- Se o erro for do tipo "vale a pena tentar de novo" (por exemplo, demorou demais ou deu uma falha passageira): o Worker espera um pouco (cada vez mais tempo entre uma tentativa e outra — é o "aguarda backoff exponencial") e tenta de novo, até um número limite de vezes.
- Se o erro for do tipo "não adianta tentar de novo" (por exemplo, o documento é inválido de um jeito que a IA sempre vai recusar): o Worker desiste na hora e marca o status como "falhou", sem ficar insistindo à toa.

6. Guardando o resultado

Quando o Provider finalmente responde com sucesso (ou depois de decidir que não vai insistir mais), o Worker manda gravar o resultado no Repositório, junto com a informação de "que versão de IA e que instrução exata foi usada" (o promptText, modelId etc.), para garantir que dá pra entender depois exatamente como aquele resultado foi gerado.

7. A decisão final — último bloco "alt"

Por fim, o sistema olha o quão confiante a IA ficou do resultado:

- Se a confiança é alta o suficiente: marca o documento como "pronto" (DONE).
- Se a confiança é baixa: marca como "esperando revisão humana" (PENDING_REVIEW).

9. Registro de Decisões de Arquitetura (ADRs)

As decisões arquiteturalmente significativas que **efetivamente moldam esta fatia vertical** (aquelas que afetam estrutura, dependências, interfaces ou técnicas de construção do código entregue) são documentadas individualmente em registros de decisão (ADRs), mantidos em docs/adr/ e versionados junto do código. Este documento não repete o raciocínio de cada decisão; referencia-o.

ID	Título	Status	Fatos/Requisitos relacionados
ADR-001	Processamento assíncrono via fila	Aceito	ENV-A, ENV-E, RF-01, RF-04, RNF-01, RNF-02
ADR-002	Arquitetura hexagonal (portas e adaptadores)	Aceito	ENV-F, RNF-03
ADR-003	Deduplicação por hash de conteúdo	Aceito	ENV-C, RF-03

Itens como o mecanismo de consumo por sistemas internos (comportamento-alvo 5) e a política de retenção de dados pessoais sensível (ENV-D) não recebem ADR nesta versão, não há decisão a registrar, apenas uma questão em aberto. Eles permanecem documentados como estão, na Seção 2.2 (fora de escopo) e na Seção 6 (riscos conhecidos e não mitigados). Se uma decisão vier a ser tomada sobre eles (inclusive com base na resposta do avaliador ao e-mail enviado em 30/08/2026) um ADR será criado retroativamente, não antes.

ADR-001 — Processamento Assíncrono via Fila

Status: Aceito

Data: 30/08/2026

Decisores: Jorge Luiz Silva Braz

Relacionado a: ENV-A, ENV-E, RF-01, RF-04, RNF-01, RNF-02 (SPEC-001)

Contexto

O comportamento-alvo do sistema exige receber um documento e devolver, em algum momento, um resultado de classificação e extração. A forma mais direta de implementar isso seria um endpoint síncrono: o cliente envia o arquivo, o servidor chama o provedor de extração e só responde depois que o resultado sai.

Essa abordagem direta esbarra em duas restrições reais do ambiente, registradas no SPEC-001:

- **ENV-A:** o provedor de extração é um modelo de linguagem multimodal de terceiro, com latência de 5 a 40 segundos por chamada, e sujeito a erro ou ausência de resposta.
- **ENV-E:** o volume médio é de 150 documentos/dia, com picos superiores a 800, concentrados numa janela de 2 horas (9h–11h).

Um endpoint síncrono, nessas condições, manteria uma conexão HTTP aberta por até 40 segundos por requisição, inviável do ponto de vista de timeout de cliente, de uso de conexões do servidor, e principalmente incompatível com o pico de volume: 800 documentos concentrados em 2 horas significam, no pior caso, dezenas de chamadas simultâneas de até 40 segundos cada, o que exigiria uma quantidade de conexões abertas simultaneamente que não escalou de forma responsável.

Além disso, uma falha do provedor durante uma chamada síncrona propagam diretamente como falha da requisição do cliente, sem espaço para retentativa transparente.

Decisão

O processamento será **desacoplado da ingestão por meio de uma fila** (BullMQ sobre Redis).

O fluxo passa a ser:

1. POST /documents faz apenas a validação de entrada (tipo real do arquivo, cálculo de hash, verificação de duplicidade) e grava o documento com status RECEIVED.
2. Um job é publicado na fila, e a API responde imediatamente ao cliente com 202 Accepted e o identificador do documento, sem aguardar a extração.
3. Um worker consome a fila de forma independente, chama o provedor de extração, aplica a lógica de retentativa (RF-04) e grava o resultado quando disponível.
4. O cliente consulta o resultado posteriormente via GET /documents/:id.

A concorrência do worker (quantos jobs processa em paralelo) é um parâmetro de configuração externo, não um valor fixo no código, isso permite absorver o pico de ENV-E ajustando um número, sem alterar lógica.

Alternativas consideradas e descartadas

Endpoint síncrono simples (chamar o provedor dentro da própria requisição HTTP)

Descartada por inviabilizar o tratamento correto de ENV-A e ENV-E, como descrito no contexto. Seria a opção mais simples de implementar, mas quebraria sob o primeiro pico real de uso, exatamente o tipo de escolha que o próprio desafio adverte que "quebra na primeira semana de uso real".

Long-polling ou WebSocket para simular resposta "quase síncrona"

Descartada por adicionar complexidade de protocolo sem resolver o problema de fundo: o gargalo não é a comunicação entre cliente e servidor, é a latência e a instabilidade do provedor de terceiro. Resolver isso na camada de transporte, e não na camada de processamento, só move o problema.

Processamento em background via setTimeout/job em memória, sem fila real (ex: sem Redis)

Descartada porque não sobrevive a um restart do processo, e não oferece nenhuma garantia de entrega, retentativa ou ordenação. Resolveria o problema em desenvolvimento

local, mas não é uma escolha honesta para um cenário que já descreve picos de volume reais.

Consequências

Positivas:

- A API de ingestão fica rápida e previsível, independentemente da lentidão ou instabilidade do provedor de terceiro (RNF-01).
- O pico de volume (ENV-E) é absorvido pela fila, que atua como amortecedor, os documentos esperam, mas o serviço não cai.
- Retentativas passam a ser responsabilidade do worker, isoladas da requisição HTTP original, uma falha transitória do provedor não vira uma falha visível para o cliente que enviou o documento.
- A fila cria um ponto natural de observabilidade: dá para medir quantos documentos estão esperando processamento em tempo real, o que não seria possível com chamadas síncronas soltas.
- Abre espaço, sem custo arquitetural adicional, para o comportamento-alvo 4 do desafio original (documento de baixa confiança fica para conferência humana) – o estado PENDING_REVIEW é só mais um destino possível depois do processamento assíncrono, não uma reestruturação.

Negativas / trade-offs assumidos:

- Introduz uma dependência de infraestrutura nova (Redis) que não existiria numa solução puramente síncrona, custo aceito conscientemente, documentado.
- O cliente não recebe mais o resultado na mesma requisição que envia o documento; precisa fazer uma segunda chamada (GET /documents/:id) para consultar. Isso exige que quem constrói sobre esta API (inclusive uma eventual Trilha B) implemente *polling* ou aguarde notificação, decisão que fica registrada como parte do contrato exposto, não como responsabilidade oculta.
- Adiciona uma fonte a mais de falha possível (a própria fila pode ficar indisponível), mitigado, mas não eliminado, pelo fato de a API continuar aceitando novas ingestões mesmo se o processamento estiver parado (RNF-01).

Diagrama de referência

Ver **Seção 8.1 (Diagrama de Componentes)** e **Seção 8.3 (Diagrama de Sequência)** do SPEC-00, este ADR é a justificativa por trás dos blocos "Fila" e "Worker de Processamento" que aparecem em ambos.

ADR-002 – Arquitetura Hexagonal (Portas e Adaptadores)

Status: Aceito

Data: 30/08/2026

Decisores: Jorge Luiz Silva Braz

Relacionado a: ENV-F, RNF-03 (SPEC-001)

Contexto

O ENV-F registra um fato do ambiente que não é hipotético, mas certo: *"o modelo do fornecedor será trocado de versão em algum momento, e os prompts vão mudar mais de uma vez ao longo do primeiro ano."* Isso não é um risco a mitigar, é uma mudança já esperada, apenas sem data definida.

Se a chamada ao provedor de extração (requisição HTTP, formato de payload, parsing da resposta) estiver espalhada diretamente dentro do Serviço de Processamento, cada troca de versão do modelo ou reformulação de prompt exige alterar código que também contém regra de negócio, misturando dois motivos de mudança independentes (decisão de negócio vs. decisão de fornecedor) na mesma classe. O mesmo raciocínio se aplica, com menor urgência, ao armazenamento de arquivo (StoragePort) e ao acesso a dados (DocumentRepository): nenhum dos dois tem fornecedor definido como imutável para o produto real, mesmo que nesta entrega ambos sejam resolvidos de forma simples.

Decisão

O domínio (Serviço de Ingestão, Serviço de Processamento, Política de Confiança) não depende diretamente de nenhuma tecnologia externa. Toda dependência externa é acessada por meio de uma porta, uma interface definida pelo próprio domínio, no vocabulário do domínio, não no vocabulário da tecnologia:

- DocumentIntelligenceProvider — abstrai a chamada ao provedor de extração.
- StoragePort — abstrai o armazenamento do arquivo original.
- DocumentRepository — abstrai a persistência dos dados do documento.

Cada porta tem, nesta entrega, exatamente um adaptador concreto (MockProvider, FileSystemStorage, PrismaPostgresRepository — ver Seção 8.1 do SPEC-001). A porta é definida pelo que o domínio *precisa*, não pelo que a tecnologia escolhida *oferece*, isto é, a assinatura do DocumentIntelligenceProvider não replica a API do provedor real; expõe apenas os métodos que o Serviço de Processamento de fato usa.

Alternativas consideradas e descartadas

Chamar o provedor de extração diretamente do Serviço de Processamento, sem interface

Mais rápido de escrever nesta entrega, mas viola diretamente o RNF-03: qualquer mudança de fornecedor, versão ou formato de resposta do ENV-F exigiria alterar a lógica de negócio junto com o código de integração. Descartada por contradizer um fato do ambiente que o próprio desafio pede explicitamente para ser tratado.

Um único repositório genérico ("Gateway") cobrindo IA, storage e banco numa classe só

Descartada por misturar responsabilidades com taxas de mudança muito diferentes: o banco de dados tende a ser estável ao longo do projeto, enquanto o provedor de IA já tem troca de versão prevista para o primeiro ano. Uma interface genérica esconderia essa diferença em vez de expor a fronteira certa.

ORM/driver do banco usado diretamente nos serviços de domínio (sem DocumentRepository)

Foi considerada como simplificação válida, já que o desafio libera de "alta cobertura de testes" e de excesso de engenharia. Descartada porque o custo de manter a porta é baixo (uma interface pequena) e o ganho é imediato para os testes de risco da Seção 10 do SPEC-001 — os testes de dedup sob concorrência e de roteamento por confiança dependem de conseguir substituir o repositório por uma versão de teste sem tocar no banco real.

Consequências

Positivas:

- A troca de provedor de IA (ENV-F) — quando acontecer, se resume a escrever um novo adaptador; o Serviço de Processamento e a Política de Confiança não são tocados.
- Os testes de risco (Seção 10 do SPEC-001) podem usar o MockProvider como implementação de teste da mesma porta usada em produção, sem duplicar contrato.
- A fronteira de cada porta documenta, por si só, "o que acontece quando uma peça precisa ser trocada".

Negativas / trade-offs assumidos — dito com honestidade:

- Nesta fatia vertical específica, a lógica de domínio por trás de cada porta ainda é pequena, o DocumentRepository, por exemplo, hoje pouco mais faz do que repassar chamadas ao Prisma. Isso é um risco reconhecido de abstração: uma porta só se paga quando protege lógica de negócio real; se o "domínio" for apenas "salvar e buscar", a interface pode se tornar uma cópia redundante do próprio client do ORM, sem ganho prático imediato, o ganho é majoritariamente sobre a porta DocumentIntelligenceProvider, cuja instabilidade (ENV-F) é real e documentada, não sobre as demais.
- Adiciona uma camada de indireção (interface + implementação) que exige mais arquivos e mais saltos de leitura do que uma chamada direta, custo aceito conscientemente em troca da substituíbilidade, e sinalizado aqui como a decisão que, entre as tomadas neste projeto, seria a mais fácil de questionar numa revisão futura.

Diagrama de referência

Ver Seção 8.1 (Diagrama de Componentes) do SPEC-001; os blocos "Portas — Interfaces do Domínio" e "Adaptadores — Implementações Concretas", e a relação tracejada "implementado por" entre eles, são a representação visual direta desta decisão.

ADR-003 — Deduplicação por Hash de Conteúdo

Status: Aceito

Data: 30/08/2026

Decisores: Jorge Luiz Silva Braz

Relacionado a: ENV-C, RF-03 (SPEC-001)

Contexto

O ENV-C registra um fato observado do ambiente real: *"o mesmo documento costuma chegar mais de uma vez, o cliente reenvia por insegurança, o atendimento reenvia por precaução."* Isso não é uma exceção rara; é comportamento esperado de quem envia documentos por WhatsApp e e-mail, sem nenhuma confirmação de recebimento clara.

Combinado ao ENV-A (cada chamada ao provedor de extração é cobrada por documento, e leva de 5 a 40 segundos), processar o mesmo documento duas vezes tem custo direto e desnecessário: dinheiro gasto em uma chamada redundante, e tempo de fila ocupado por um trabalho que já foi feito. Além disso, sem uma forma de identificar reenvio, o sistema poderia gerar dois registros distintos para o mesmo documento físico, confundindo qualquer consulta posterior (GET /documents) sobre quantos documentos foram, de fato, recebidos.

O ENV-B agrava o problema: não há validação alguma do lado de quem envia, e os nomes de arquivo são inconsistentes ("WhatsApp Image 2026-08-11 at 09.12.33.jpeg", "scan0001.pdf"), ou seja, não é possível usar nome de arquivo como critério de identidade do documento, porque o mesmo conteúdo pode chegar com nomes diferentes, e nomes iguais não garantem o mesmo conteúdo.

Decisão

Todo documento recebido tem um hash criptográfico SHA-256 calculado sobre os bytes brutos do arquivo, no momento da ingestão, antes de qualquer outra operação. Esse hash é armazenado com uma constraint de unicidade no banco de dados, e a verificação de existência acontece de forma transacional, dentro da mesma operação que grava o novo documento, não como uma leitura seguida de uma escrita separada.

Se o hash já existe, a API responde com o documento existente (200, status atual) e não enfileira novo processamento. Se o hash é novo, o fluxo segue normalmente (grava com status RECEIVED, enfileira — ver ADR-001).

A verificação "existe + grava se não existir" é tratada como uma única operação atômica no banco (constraint de unicidade + tratamento de violação, não um SELECT seguido de um INSERT em passos separados), para que duas ingestões do mesmo conteúdo, disparadas ao mesmo tempo, não resultem em duas chamadas ao provedor de extração, esse é, inclusive, o critério de aceitação do teste descrito na Seção 10.1 do SPEC-001.

Alternativas consideradas e descartadas

Deduplicação por nome de arquivo

Descartada de imediato: o ENV-B deixa explícito que nomes são inconsistentes e não confiáveis, o mesmo documento pode chegar com nomes diferentes, e essa via nem resolveria o problema que motiva a decisão.

Deduplicação por par (remetente, timestamp aproximado)

Descartada por depender de metadado de origem que o desafio não garante existir de forma confiável nesta fatia (não há autenticação real de quem envia, ver Seção 2.2 do SPEC-001), e por ser uma heurística aproximada quando uma verificação exata (hash) é possível e mais barata computacionalmente.

Verificação de existência antes de gravar, sem constraint no banco (chechar em código, depois inserir)

Foi a primeira abordagem cogitada, por ser mais simples de implementar. Descartada porque não é segura sob concorrência: duas requisições simultâneas do mesmo conteúdo poderiam ambas checar "não existe" antes que qualquer uma tivesse gravado, resultando em dois registros e duas chamadas pagas ao provedor, exatamente a falha que o fato ENV-C e o custo do ENV-A tornam inaceitável. A constraint de unicidade no banco move essa garantia para a camada onde ela é realmente atômica.

Deduplicação perceptual (hash de similaridade de imagem, tolerante a recompressão/rotação/ângulo)

Reconhecida como mais próxima do problema real do escritório, uma foto tirada duas vezes do mesmo papel gera bytes diferentes, então o hash exato não pega esse caso. Descartada nesta entrega, mas não por desconhecimento do método: ver análise técnica completa abaixo.

Nota técnica: por que a deduplicação perceptual não é trivial

Existe uma abordagem estabelecida para esse problema, hash perceptual (ex.: pHash, dHash, aHash), que reduz uma imagem a uma representação simplificada (baixa resolução, escala de cinza, resumo de gradientes de brilho) e compara hashes por distância, não por igualdade exata. Duas fotos do mesmo papel físico, mesmo com variação de ângulo ou iluminação, tendem a gerar hashes próximos.

A decisão de não adotá-la nesta entrega não é por ausência de técnica conhecida, mas porque implementá-la com segurança introduz três problemas que a deduplicação exata não tem:

1. Calibração do limiar de similaridade, a distância entre hashes que define "mesmo documento" não vem de fórmula; exige ajuste empírico contra volume real de documentos do escritório, que não está disponível nesta fase.
2. Risco de falso positivo com custo assimétrico, ao contrário do hash exato (sem ambiguidade possível), o hash perceptual pode classificar como duplicata dois documentos de pessoas diferentes fotografados em condições semelhantes. Num sistema que trata dado pessoal sensível (ENV-D), esse erro é mais grave do que o problema que a deduplicação pretende evitar (reprocessamento redundante), mistura de dados entre titulares distintos.
3. PDFs exigem uma etapa adicional, hash perceptual opera sobre imagem; um PDF precisaria ser renderizado como imagem antes da comparação, adicionando uma dependência e uma nova fonte de falha ao pipeline de ingestão.

Diante disso, a decisão registrada é: manter a deduplicação exata (SHA-256) nesta entrega, e tratar quase-duplicatas como risco conhecido e documentado (RISK-04, Seção 6 do SPEC-001), em vez de implementar uma solução parcial e não calibrada, cujo comportamento sob falso positivo seria mais arriscado do que a ausência da funcionalidade.

Consequências

Positivas:

- Reenvios idênticos (o caso mais comum descrito no ENV-C, reenvio por insegurança do mesmo arquivo) não geram custo adicional de processamento nem registros duplicados.
- A garantia é forte porque vive na constraint do banco, não em lógica de aplicação que poderia ser burlada por uma corrida de concorrência.
- O teste da Seção 10.1 do SPEC-001 pode validar essa garantia de forma determinística, sem depender de sorte de timing.

Negativas / limitação conhecida – dito com honestidade:

- Esta deduplicação só detecta bytes idênticos. Uma foto tirada duas vezes do mesmo documento físico, com ângulo, iluminação ou compressão levemente diferentes, gera hashes diferentes e não é detectada como duplicata, apenas reduzida a "mais um documento processado normalmente". Este comportamento está formalmente registrado como RISK-04 na Seção 6 do SPEC-001, com a análise técnica completa documentada acima.
- A chave de hash cresce indefinidamente no banco se nunca for expurgada, não há TTL definido nesta entrega; fica registrado como ponto em aberto, ligado à mesma pendência de política de retenção (RISK-02) já citada na Seção 6 do SPEC-001.

Diagrama de referência

Ver Seção 8.3 (Diagrama de Sequência) do SPEC-001 — o bloco alt [hash já existe] / [hash novo], logo após "calcula hash SHA-256", é a representação direta desta decisão em execução.